

Colorado Pollen Detection: Methods and Results

Table of contents

2. Materials and Methods	1
2.1. Image Preprocessing and Tiling Strategy	1
2.2. Two-Stage Deep Learning Architecture	2
2.3. Continuous Active Learning and Annotation	2
2.4. Computational Environment and Inference	2
3. Results	3
Verifying Annotations	3
Remote Storage	3
Species Dataset Configuration	3

2. Materials and Methods

2.1. Image Preprocessing and Tiling Strategy

High-resolution microscopy scans were acquired in Zeiss .czi format. Due to the gigapixel dimensions of the raw slides, which exceed the memory constraints of modern convolutional neural networks, a deterministic tiling algorithm was employed. First, a Maximum Intensity Projection (MIP) was applied to the Z-stacks to collapse the three-dimensional volumetric data into a two-dimensional representation while preserving the structural topography of the pollen grains.

The resulting macroscopic images were dynamically partitioned into overlapping 640×640 pixel tiles. Overlap was strictly maintained to prevent truncation of pollen grains intersecting the grid boundaries. To suppress false-positive detections caused by inorganic debris, air bubbles, and slide artifacts, purely empty background tiles were explicitly sampled and injected into

the training dataset as “Hard Negatives” (tiles with image data but lacking bounding box labels).

2.2. Two-Stage Deep Learning Architecture

To maximize both detection recall on heterogeneous biological backgrounds (such as messy stigmas) and taxonomic classification precision, the detection pipeline was bifurcated into a two-stage cascaded architecture using the Ultralytics YOLOv8 framework.

Stage 1: General Pollen Instance Segmentation A YOLOv8-Large Instance Segmentation model (`yolo8l-seg`) was trained to detect any biological structure morphologically consistent with a pollen grain. During this stage, all species-specific class labels were collapsed into a singular `pollen` class (Class ID 0). This stage prioritized high-recall boundary detection over taxonomy. The model was trained for 500 epochs with dynamic augmentations including HSV color perturbation, multi-scale mosaic, and mixup to induce robustness against overlapping structures.

Stage 2: Species-Specific Classification Following Stage 1 inference, the identified polygon boundaries were utilized to dynamically crop the pollen grains from the source image. These standardized, isolated crops were then fed into a secondary YOLOv8-Extra-Large Image Classification model (`yolo8x-cls`). This model was exclusively trained on pure, manually curated reference pollen (e.g., *Ranunculus adoneus*) extracted directly from pristine anther scans. This two-stage decoupling prevents the classification model from learning irrelevant stigma background noise.

2.3. Continuous Active Learning and Annotation

To continuously adapt the model to novel pollen orientations and previously unseen biological debris, a human-in-the-loop Active Learning interface was developed. The system allows domain experts to rapidly triage Stage 1 predictions on novel data: 1. **Acceptance:** Perfect segmentations are committed immediately to the `Curated_Retrain_Data` repository. 2. **Hard Negatives:** Erroneous detections (e.g., misidentified debris) are rejected and stored with empty annotation matrices to explicitly penalize the model in future training cycles. 3. **Manual Refinement:** Imprecise polygons are passed through a data pipeline to Roboflow, where experts manually redraw the physical boundaries using semantic polygon tools before reintegrating them into the training corpus.

2.4. Computational Environment and Inference

Model training and heavy preprocessing workloads were orchestrated as distributed containerized jobs via Kubernetes on the CESNET MetaCentrum High-Performance Computing cluster, utilizing NVIDIA A100 and A40 Tensor Core GPUs. Post-inference, the pipeline reconstructs

the 640×640 predictions into downsampled 2000-pixel macro-stitches with superimposed polygon masks for rapid visual verification, alongside tabular CSV exports containing spatial coordinates and morphological area (μm^2) metrics for each detected grain.

3. Results

Verifying Annotations

To confirm the quality of the pseudo-labeled segmentations, use the `visualize_labels.py` script:

```
python src/visualize_labels.py \  
  --images ./dataset_ran_ado/train/images \  
  --labels ./dataset_ran_ado/train/labels \  
  --out ./verification_viz \  
  --num 20
```

This will generate 20 random samples in `./verification_viz` with the segmentation polygons overlaid on the tiles. Inspect these images to ensure: 1. Polygons tightly fit the pollen grains. 2. The species classification is correct. 3. No significant background noise is being labeled.

Remote Storage

Full datasets and models are stored on CESNET S3: - **Bucket:** bucket - **Datasets:** PEG/Colorado/Staged_area/ - **Detections:** PEG/Colorado/Detected/ - **Models:** PEG/Colorado/trained_models/

Species Dataset Configuration

To give users full dynamic control over the massive Multi-Species model index, the dataset taxonomy has been completely decoupled from the codebase into an external configuration manifest.

You can freely inject, override, or classify new species targets (e.g. `Gen_alg`) by editing the `src/species_manifest.csv` file mapping directly inside this repository.

The Kubernetes pipeline seamlessly syncs this mapping table upon build and natively routes CZI binary processing identically to the table rules without needing deep-code interventions.